

```
!pip install gensim
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.3.3)
Requirement already satisfied: numpy<2.0,>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.26.4)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.13.1)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.0.5)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1) (1.17.2)
```

```
!pip install pyLDAvis
```

```
Collecting pyLDAvis
```

```
  Downloading pyLDAvis-3.4.1-py3-none-any.whl.metadata (4.2 kB)
```

```
Requirement already satisfied: numpy>=1.24.2 in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (1.13.1)
Requirement already satisfied: pandas>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (2.2.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (1.4.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (3.1.4)
Requirement already satisfied: NumExpr in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (2.10.2)
Collecting funcy (from pyLDAvis)
```

```
  Downloading funcy-2.0-py2.py3-none-any.whl.metadata (5.9 kB)
```

```
Requirement already satisfied: scikit-learn>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (1.5.2)
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (4.3.3)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (75.1.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pyLDAvis) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.0.0) (3.5.0)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.0.5)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2) (3.0.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil) (1.17.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1) (1.17.2)
Downloading pyLDAvis-3.4.1-py3-none-any.whl (2.6 MB)
```

```
2.6/2.6 MB 35.5 MB/s eta 0:00:00
```

```
Downloading funcy-2.0-py2.py3-none-any.whl (30 kB)
```

```
Installing collected packages: funcy, pyLDAvis
```

```
Successfully installed funcy-2.0 pyLDAvis-3.4.1
```

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Gensim
import gensim
import gensim.corpora as corpora
from gensim.utils import simple_preprocess
from gensim.models import CoherenceModel
from pprint import pprint

# Plotting tools
import pyLDAvis
import pyLDAvis.gensim_models as gensimvis

import warnings

warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
import nltk
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

##Topic Modelling: Topic modeling is an unsupervised learning technique used to extract topics from a large corpus of text automatically. Topics can be defined as the co-occurring terms that are frequently repeating in the text corpus. Topic modeling is helpful in various tasks such as clustering documents, understanding and summarizing an extensive collection of textual documents and retrieving information from an extensive collection of data, etc. In this notebook I will perform topic modelling using LDA(Latent Dirichlet Allocation) and BERT.

Here I am using medium article dataset to perform Topic Modelling. The dataset consist 350 articles related to AI, Machine Learning and Data Science.

✖ Exploring the dataset :

```
medium_articles = pd.read_csv("/content/articles.csv")
medium_articles.head()
```

	author	claps	reading_time	link	title	te
0	Justin Lee	8.3K	11	https://medium.com/swlh/chatbots-were-the-next...	Chatbots were the next big thing: what happened...	Oh, how t headlin blared:\nChatbc were T
1	Conor Dewey	1.4K	7	https://towardsdatascience.com/python-for-data...	Python for Data Science: 8 Concepts You May Ha...	If you've ev found yours looking up the s
2	William Koehrsen	2.8K	11	https://towardsdatascience.com/automated-featu...	Automated Feature Engineering in Python – Towa...	Machine learni is increasing moving from h
3	Gant Laborde	1.3K	7	https://medium.freecodecamp.org/machine-learni...	Machine Learning: how to go from Zero to Hero ...	If yc understanding A.I. and Machi Lea
4	Emmanuel Ameisen	935	11	https://blog.insightdatascience.com/reinforcem...	Reinforcement Learning from scratch – Insight ...	Want to lea about appli Artificial Intell

```
article_text = medium_articles.text

#total number of author
print(f"Total Number of unique authors : {len(medium_articles.author.unique())}")

#counting the articles of different authors
article_counts = dict()
for author in medium_articles.author.unique():
    article_counts[author] = medium_articles[medium_articles.author == author].text
```

```
for author in medium_articles.author:
    article_count = len(medium_articles[medium_articles["author"]==author])
    article_counts[author] = article_count

#sorting the article_counts on the basis of article count
article_counts = dict(sorted(article_counts.items(), key=lambda x: x[1], reverse=True))

#plot histogram of top 10 authors and their article counts
plt.figure(figsize=(14,7))
plt.subplot(1, 2, 1)
sns.barplot(x = list(article_counts.keys())[:10],y=list(article_counts.values())[:10])
plt.title("Authors and Number of Articles")
plt.xlabel("Authors")
plt.ylabel("No. of Articles")
plt.xticks(rotation = 90)

#counting the total number of claps for each author
def convert_to_num(clap_count):
    if "K" in clap_count:
        if "." in clap_count:
            clap_count = re.sub("\.", "", clap_count[:-1])+"00"
        else:
            clap_count = clap_count[:-1]+"000"
    return(int(clap_count))

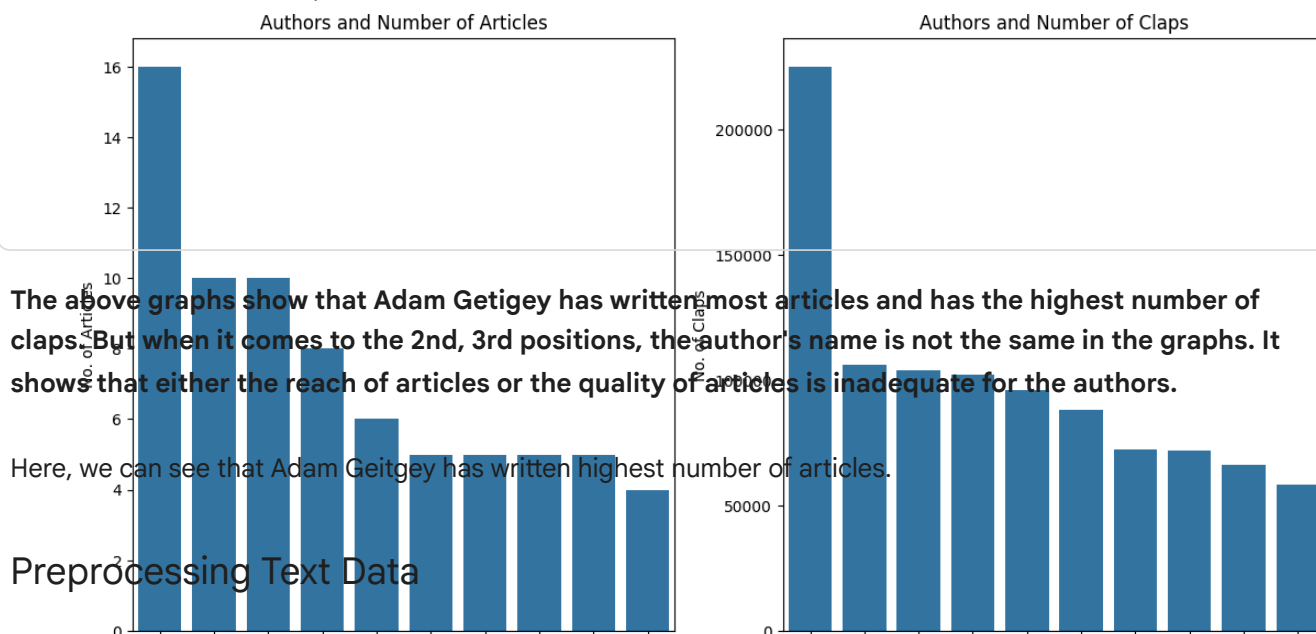
medium_articles.claps = medium_articles.claps.apply(convert_to_num)

#counting total number of claps for each author
clap_counts = dict()
for author in medium_articles.author:
    clap_count = medium_articles[medium_articles["author"]==author]["claps"]
    clap_counts[author] = sum(clap_count)

#sort clap_counts on the basis of counting of claps
clap_counts = dict(sorted(clap_counts.items(), key=lambda x: x[1], reverse=True))

#plot histogram of top 10 authors and their article counts
plt.subplot(1, 2, 2)
sns.barplot(x = list(clap_counts.keys())[:10],y=list(clap_counts.values())[:10])
plt.title("Authors and Number of Claps")
plt.xlabel("Authors")
plt.ylabel("No. of Claps")
plt.xticks(rotation = 90)
plt.show()
```

```
<>:32: SyntaxWarning: invalid escape sequence '\.'
<>:32: SyntaxWarning: invalid escape sequence '\.'
/tmp/ipython-input-3300996156.py:32: SyntaxWarning: invalid escape sequence '\.'
  clap_count = re.sub("\.", "", clap_count[:-1])+"00"
Total Number of unique authors : 182
```



Preprocessing Text Data

```
#lowercase the articles
medium_articles.text = medium_articles.text.apply(lambda t : t.lower())
```

```
# Dictionary of English Contractions
```

```
contractions_dict = { "ain't": "are not", "is": "is", "aren't": "are not",
  "can't": "cannot", "can't've": "cannot have",
  "'cause": "because", "could've": "could have", "couldn't": "could not",
  "couldn't've": "could not have", "didn't": "did not", "doesn't": "does not",
  "don't": "do not", "hadn't": "had not", "hadn't've": "had not have",
  "hasn't": "has not", "haven't": "have not", "he'd": "he would",
  "he'd've": "he would have", "he'll": "he will", "he'll've": "he will have",
  "how'd": "how did", "how'd'y": "how do you", "how'll": "how will",
  "I'd": "I would", "I'd've": "I would have", "I'll": "I will",
  "I'll've": "I will have", "I'm": "I am", "I've": "I have", "isn't": "is not",
  "it'd": "it would", "it'd've": "it would have", "it'll": "it will",
  "it'll've": "it will have", "let's": "let us", "ma'am": "madam",
  "mayn't": "may not", "might've": "might have", "mightn't": "might not",
  "mightn't've": "might not have", "must've": "must have", "mustn't": "must not",
  "mustn't've": "must not have", "needn't": "need not",
  "needn't've": "need not have", "o'clock": "of the clock", "oughtn't": "ought not",
  "oughtn't've": "ought not have", "shan't": "shall not", "shan't've": "shall not",
  "shan't've": "shall not have", "she'd": "she would", "she'd've": "she would have",
  "she'll": "she will", "she'll've": "she will have", "should've": "should have",
  "shouldn't": "should not", "shouldn't've": "should not have", "so've": "so have",
  "that'd": "that would", "that'd've": "that would have", "there'd": "there would",
  "there'd've": "there would have", "they'd": "they would",
  "they'd've": "they would have", "they'll": "they will",
  "they'll've": "they will have", "they're": "they are", "they've": "they have",
  "to've": "to have", "wasn't": "was not", "we'd": "we would",
  "we'd've": "we would have", "we'll": "we will", "we'll've": "we will have",
  "we're": "we are", "we've": "we have", "weren't": "were not", "what'll": "what will",
  "what'll've": "what will have", "what're": "what are", "what've": "what have",
  "when've": "when have", "where'd": "where did", "where've": "where have",
  "who'll": "who will", "who'll've": "who will have", "who've": "who have",
  "why've": "why have", "will've": "will have", "won't": "will not",
  "won't've": "will not have", "would've": "would have", "wouldn't": "would not",
  "wouldn't've": "would not have", "y'all": "you all", "y'all'd": "you all would",
  "y'all'd've": "you all would have", "y'all're": "you all are",
  "y'all've": "you all have", "you'd": "you would", "you'd've": "you would have",
  "you'll": "you will", "you'll've": "you will have", "you're": "you are",
  "you've": "you have" }
```

```
# Regular expression for finding contractions
contractions_re=re.compile('%s' % '|'.join(contractions_dict.keys()))

# Function for expanding contractions
def expand_contractions(text,contractions_dict=contractions_dict):
    def replace(match):
        return contractions_dict[match.group(0)]
    return contractions_re.sub(replace, text)

# Expanding Contractions in the text data
medium_articles.text = medium_articles.text.apply(lambda x:expand_contractions(x))
```

Removing Stopwords and Punctuations: Stopwords are the words that are highly occurred in a language; they do not add any significance to the sentence. "a," "an," "the," "in" are some examples of stopwords. So we generally ignore stop words during our NLP task. In some NLP tasks, stop words are also important, but stopwords are unnecessary for topic modeling. We also remove punctuation marks because they are also unnecessary and do not contribute anything.

```
print(stopwords.words('english'))
```

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'a
```

```
print(f"These are the pucntions which string.punctuation consist : {string.punctuation}")
```

```
These are the pucntions which string.punctuation consist : !"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

```
stop_words = set(stopwords.words('english'))

def remove_stopwords(article):
    "Return the articel after remvoing stopwords"
    article_tokens = word_tokenize(article)
    filtered_article = [word for word in article_tokens if not word in stop_words]
    return " ".join(filtered_article)

#removing stopwords
medium_articles.text = medium_articles.text.apply(remove_stopwords)

#removing Punctuations
medium_articles.text = medium_articles.text.apply(lambda x: re.sub('[%s]' % re.escape(string.punctuation), '', x))

#removing digits
medium_articles.text = medium_articles.text.apply(lambda x: re.sub('\w*\d\w*', '', x))

medium_articles.text[50]
```

```
<>:17: SyntaxWarning: invalid escape sequence '\w'
<>:17: SyntaxWarning: invalid escape sequence '\w'
/tmp/ipython-input-2678638626.py:17: SyntaxWarning: invalid escape sequence '\w'
    medium_articles.text = medium_articles.text.apply(lambda x: re.sub('\w*\d\w*', '', x))
'upd april      scikit flow merged tensorflow since version called tensorflow learn tflearn google
released machine learning framework called tensorflow ' taking world storm stars github lot public
ity general excitement ai researchers use something regular problem data scientist may ai researc
her - build interesting problems time reasonable question data scientist already number tools too
lbox r scikit learn etc care yet another framework answer two part let ' start simple example
- take titanic dataset kaggle first make sure installed tensorflow scikit learn helpful libs inclu
ding scikit flow simplifying lot work tensorflow get dataset code http githubcomilblackdragontfexam
ples quick look data use ipython ipython notebook ease interactive exploration let ' test predict
survived class based float variables scikit learn separate dataset features target fill na data ze
ros build logistic regression predicting training data gives us measure accuracy cause ' ...'
```

There are still some marks like quotation marks, hyphen and apostrophe still remaining in the article, we also need to remove them.

```
def remove_extra_marks(article):
    extra_keys = ["'", "-", "'", '"']
    article_tokens = word_tokenize(article)
    filtered_article = [word for word in article_tokens if not word in extra_keys]
    return " ".join(filtered_article)

medium_articles.text = medium_articles.text.apply(remove_extra_marks)
```

```
#printing arbitrary example to visulise clean data
medium_articles.text[50]
```

```
'upd april scikit flow merged tensorflow since version called tensorflow learn tflearn google release
d machine learning framework called tensorflow taking world storm stars github lot publicity general
excitement ai researchers use something regular problem data scientist may ai researcher build intere
sting problems time reasonable question data scientist already number tools toolbox r scikit learn et
c care yet another framework answer two part let start simple example take titanic dataset kaggle fir
st make sure installed tensorflow scikit learn helpful libs including scikit flow simplifying lot wor
k tensorflow get dataset code http githubcomilblackdragontfexamples quick look data use ipython ipyth
on notebook ease interactive exploration let test predict survived class based float variables scikit
learn separate dataset features target fill na data zeros build logistic regression predicting traini
ng data gives us measure accuracy cause properly evaluate model quality test dataset used s...'
```

Lemmatisation: Lemmatization process Topic modeling algorithm to avoid making mistakes by converting the word in its root from like lemmatization prevents counting "help", "helps", "helping" as three different words. Stemming can also be used, but it performs the task more forcefully, and sometimes the word loses its meaning, so I prefer lemmatization for the topic modeling task.

Here I am using **spacy** library to perform the lemmatization task.

```
import spacy
nlp = spacy.load('en_core_web_sm')
print(f"\"helps\" after lemmatization : {nlp(\"helps\")[0].lemma_}")
print(f"\"helping\" after lemmatization : {nlp(\"helping\")[0].lemma_}")
```

```
"helps" after lemmatization : help
"helping" after lemmatization : help
```

```
def lemmatize(text):
    """Return text after performing the lemmatization"""
    doc = nlp(text)
    tokens = [token for token in doc]
    return " ".join([token.lemma_ for token in doc])

#lemmatize the articles
medium_articles.text = medium_articles.text.apply(lemmatize)
```

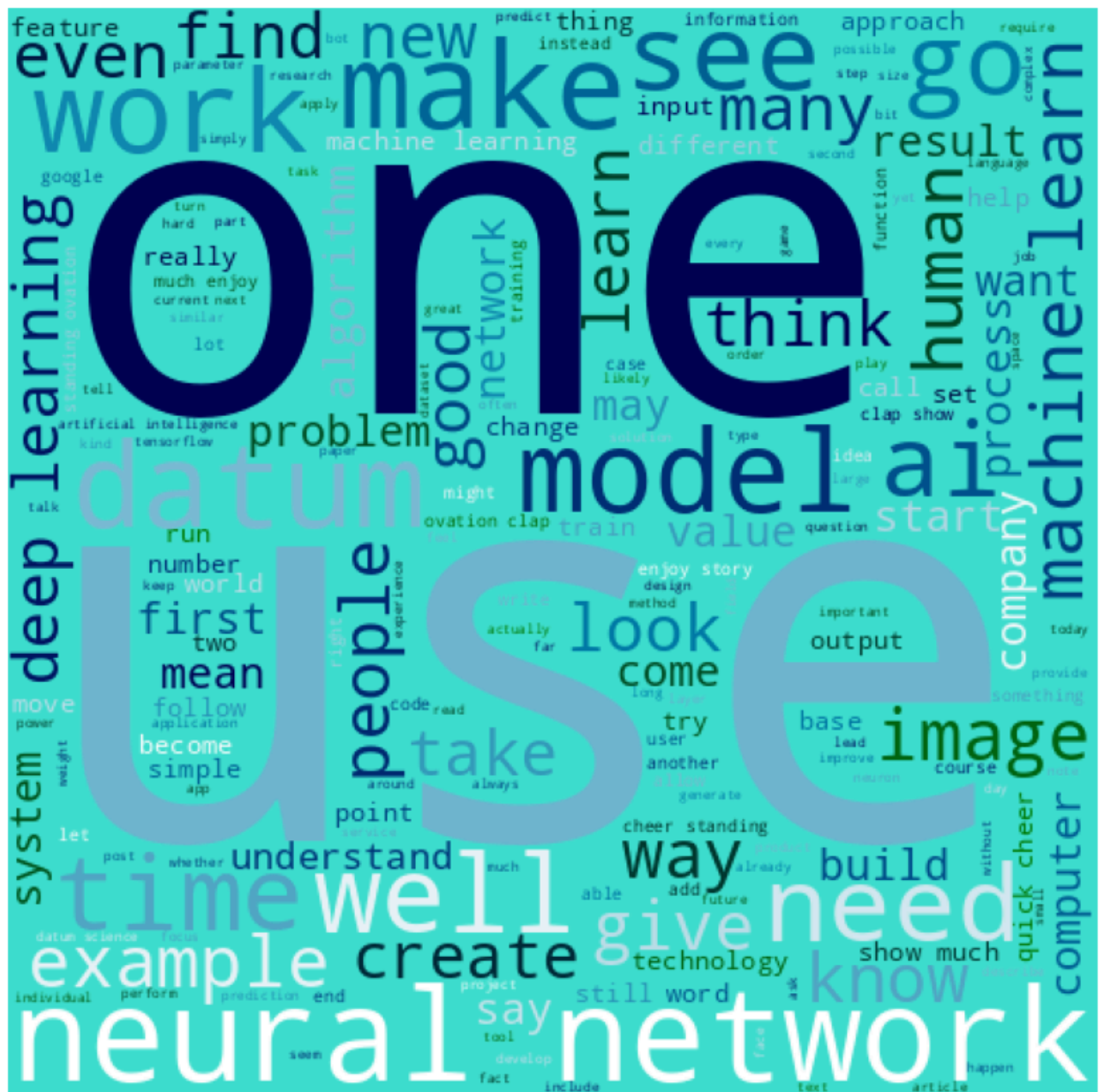
Now we have finished with all the data preprocessing steps, and let's plot a word cloud to visualize the most frequently used words and see the most commonly occurred word in the articles.

```
import wordcloud
from wordcloud import WordCloud

#combine all the articles
article_data = ""
for article in medium_articles.text:
    article_data = article_data+" "+article

#ploting the word cloud
plt.figure(figsize=(10, 10))
```

```
wordcloud = WordCloud(width = 500, height = 500, background_color='#40E0D0', colormap="ocean", random_state=42)
plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```



```
#tokenize articles
tokenize_article = medium_articles.text.apply(lambda x : x.split())
id2word = corpora.Dictionary(tokenize_article)

# Create Corpus
texts = tokenize_article

# Term Document Frequency
corpus = [id2word.doc2bow(text) for text in texts]

#printing 50 words from the text corpus
corpus_example = [(id2word[id], freq) for id, freq in cp] for cp in corpus[:2]]
corpus_example[0][:50]

[("", 8),
 ('able', 1),
 ('abruptly', 1),
 ('acceptance', 1),
 ('access', 1),
```

```
# build LDA model for 10 topic
lda_model = gensim.models.ldamodel.LdaModel(corpus=corpus,
                                             id2word=id2word,
                                             num_topics=10,
                                             random_state=100,
                                             update_every=1,
                                             chunksize=100,
                                             passes=10,
                                             alpha='symmetric',
                                             per_word_topics=True,
                                             eta = 0.6)
```

```
[0,
'0.006*"genome" + 0.005*"pacman" + 0.005*"craig" + 0.005*"specie" + '
'0.005*"analytica" + 0.004*"blockchain" + 0.004*"trump" + 0.003*"trust" + '
'0.003*"election" + 0.003*"cambridge"'),
(1,
'0.008*"action" + 0.006*"move" + 0.006*"game" + 0.006*"value" + '
'0.005*"human" + 0.005*"policy" + 0.005*"use" + 0.005*"agent" + '
'0.004*"neuron" + 0.004*"process"'),
(2,
'0.021*"de" + 0.009*"en" + 0.007*"que" + 0.007*"la" + 0.007*"para" + '
```



```

'0.005*"el" + 0.005*"como" + 0.005*"es" + 0.004*"con" + 0.004*"se"'),
(3,
'0.014*"machine" + 0.013*"learn" + 0.013*"course" + 0.007*"https" + '
'0.007*"python" + 0.006*"cheat" + 0.006*"sheet" + 0.006*"datum" + '
'0.005*"review" + 0.005*"learning"'),
(4,
'0.012*"deep" + 0.009*"learning" + 0.007*"marcus" + 0.006*"system" + '
'0.004*"f" + 0.004*"m" + 0.003*"say" + 0.003*"arxiv" + 0.003*"g" + '
'0.003*"sort"'),
(5,
'0.016*"network" + 0.013*"use" + 0.011*"neural" + 0.009*"model" + '
'0.008*"learn" + 0.008*"one" + 0.007*"datum" + 0.007*"layer" + 0.006*"image" '
'+ 0.006*"like"'),
(6,
'0.010*"de" + 0.006*"la" + 0.006*"l" + 0.005*"et" + 0.004*"les" + 0.004*"le" '
'+ 0.004*"des" + 0.003*"à" + 0.003*"que" + 0.003*"une"'),
(7,
'0.015*"rcnn" + 0.007*"cnn" + 0.007*"region" + 0.006*"box" + 0.006*"image" + '
'0.006*"fast" + 0.005*"object" + 0.005*"title" + 0.004*"mask" + '
'0.004*"proposal"'),
(8,
'0.010*"table" + 0.008*"action" + 0.006*"reward" + 0.006*"state" + '
'0.005*"cluster" + 0.005*"qlearn" + 0.004*"qvalue" + 0.004*"give" + '
'0.004*"vector" + 0.004*"environment"'),
(9,
'0.007*"ai" + 0.005*"people" + 0.005*"like" + 0.005*"human" + '
'0.005*"machine" + 0.005*"make" + 0.005*"one" + 0.005*"use" + '
'0.004*"intelligence" + 0.004*"system"')]

```

Visualising Topics:

```

# Visualize the topics
pyLDAvis.enable_notebook()
vis = gensimvis.prepare(lda_model, corpus, id2word)
vis

```

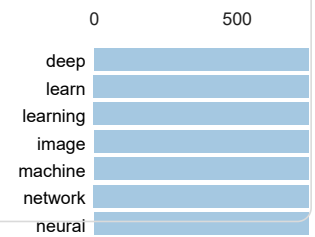
Selected Topic:

Slide to adjust relevance

$\lambda = 1$

Intertopic Distance Map (via multidimensional scaling)

PC2

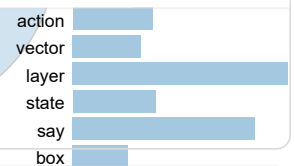


```
# Compute Perplexity
print('\nPerplexity : ', lda_model.log_perplexity(corpus))
```

```
# Compute Coherence Score
coherence_model_lda = CoherenceModel(model=lda_model, texts=tokenize_article, dictionary=id2word, coherence_model_lda = coherence_model_lda.get_coherence())
print('\nCoherence Score: ', coherence_lda)
```

Perplexity : -7.812254874305959

Coherence Score: 0.4894567031815618



```
def calculate_coherence_score(n, alpha, beta):
    lda_model = gensim.models.ldamodel.LdaModel(corpus=corpus,
                                                id2word=id2word,
                                                num_topics=n,
                                                random_state=100,
                                                update_every=1,
                                                chunksize=100,
                                                passes=10,
                                                alpha=alpha,
                                                per_word_topics=True,
                                                eta = beta)

    coherence_model_lda = CoherenceModel(model=lda_model, texts=tokenize_article, dictionary=id2word, coherence_model_lda = coherence_model_lda.get_coherence())
    coherence_lda = coherence_model_lda.get_coherence()
    return coherence_lda

#list containing various hyperparameters
no_of_topics = [2,5,7,10,12,14]
alpha_list = ['symmetric',0.3,0.5,0.7]
beta_list = ['auto',0.3,0.5,0.7]
```

```
for n in no_of_topics:
    for alpha in alpha_list:
        for beta in beta_list:
            coherence_score = calculate_coherence_score(n, alpha, beta)
            print(f'n : {n} ; alpha : {alpha} ; beta : {beta} ; Score : {coherence_score}')
```

```

n : 10 ; alpha : 0.3 ; beta : 0.3 ; Score : 0.4492669448839549
n : 10 ; alpha : 0.3 ; beta : 0.5 ; Score : 0.4579377368860804
n : 10 ; alpha : 0.3 ; beta : 0.7 ; Score : 0.5233551362512975
n : 10 ; alpha : 0.5 ; beta : auto ; Score : 0.3891394585187494
n : 10 ; alpha : 0.5 ; beta : 0.3 ; Score : 0.44638459306178097
n : 10 ; alpha : 0.5 ; beta : 0.5 ; Score : 0.46212002292357796
n : 10 ; alpha : 0.5 ; beta : 0.7 ; Score : 0.523787436859047
n : 10 ; alpha : 0.7 ; beta : auto ; Score : 0.37758408673972005
n : 10 ; alpha : 0.7 ; beta : 0.3 ; Score : 0.4383187224856079
n : 10 ; alpha : 0.7 ; beta : 0.5 ; Score : 0.46956377445911457
n : 10 ; alpha : 0.7 ; beta : 0.7 ; Score : 0.5231698009068972
n : 12 ; alpha : symmetric ; beta : auto ; Score : 0.38936724220168184
n : 12 ; alpha : symmetric ; beta : 0.3 ; Score : 0.4850613755027293
n : 12 ; alpha : symmetric ; beta : 0.5 ; Score : 0.41445127158934003
n : 12 ; alpha : symmetric ; beta : 0.7 ; Score : 0.5199470493404631
n : 12 ; alpha : 0.3 ; beta : auto ; Score : 0.3882265628001684
n : 12 ; alpha : 0.3 ; beta : 0.3 ; Score : 0.46632444943226964
n : 12 ; alpha : 0.3 ; beta : 0.5 ; Score : 0.44783417924872815
n : 12 ; alpha : 0.3 ; beta : 0.7 ; Score : 0.49567051350803953
n : 12 ; alpha : 0.5 ; beta : auto ; Score : 0.3856149494025898
n : 12 ; alpha : 0.5 ; beta : 0.3 ; Score : 0.4628178482058098
n : 12 ; alpha : 0.5 ; beta : 0.5 ; Score : 0.442318225857141
n : 12 ; alpha : 0.5 ; beta : 0.7 ; Score : 0.48828030531977773
n : 12 ; alpha : 0.7 ; beta : auto ; Score : 0.3831323165889556
n : 12 ; alpha : 0.7 ; beta : 0.3 ; Score : 0.45997118671194576
n : 12 ; alpha : 0.7 ; beta : 0.5 ; Score : 0.4467421322031542
n : 12 ; alpha : 0.7 ; beta : 0.7 ; Score : 0.49559168358196953
n : 14 ; alpha : symmetric ; beta : auto ; Score : 0.44001989599493685
n : 14 ; alpha : symmetric ; beta : 0.3 ; Score : 0.44301946891374283
n : 14 ; alpha : symmetric ; beta : 0.5 ; Score : 0.41180722835769445
n : 14 ; alpha : symmetric ; beta : 0.7 ; Score : 0.4070594344303206
n : 14 ; alpha : 0.3 ; beta : auto ; Score : 0.4393083531821976
n : 14 ; alpha : 0.3 ; beta : 0.3 ; Score : 0.42993950238049544
n : 14 ; alpha : 0.3 ; beta : 0.5 ; Score : 0.4090529907977786
n : 14 ; alpha : 0.3 ; beta : 0.7 ; Score : 0.4209295029398575
n : 14 ; alpha : 0.5 ; beta : auto ; Score : 0.4315328114363849
n : 14 ; alpha : 0.5 ; beta : 0.3 ; Score : 0.43704958173914915
n : 14 ; alpha : 0.5 ; beta : 0.5 ; Score : 0.4376312072885182
n : 14 ; alpha : 0.5 ; beta : 0.7 ; Score : 0.493290074773103
n : 14 ; alpha : 0.7 ; beta : auto ; Score : 0.436152484280402
n : 14 ; alpha : 0.7 ; beta : 0.3 ; Score : 0.45541988456530763
n : 14 ; alpha : 0.7 ; beta : 0.5 ; Score : 0.4259108324191119
n : 14 ; alpha : 0.7 ; beta : 0.7 ; Score : 0.5086389262289019

```

The highest coherence score is : 0.4962 for number of topics = 14 alpha = 0.3 and beta = auto

Final LDA Model :

```

n = 14
alpha = 0.3
beta = "auto"
lda_model = gensim.models.ldamodel.LdaModel(corpus=corpus,
                                             id2word=id2word,
                                             num_topics=n,
                                             random_state=100,
                                             update_every=1,
                                             chunksize=100,
                                             passes=10,
                                             alpha=alpha,
                                             per_word_topics=True,
                                             eta = beta)

coherence_model_lda = CoherenceModel(model=lda_model, texts=tokenize_article, dictionary=id2word, coherence_model_lda = coherence_model_lda.get_coherence())
print('\nCoherence Score: ', coherence_lda)

```

Coherence Score: 0.4393083531821976

```

# Visualize the topics
pyLDAvis.enable_notebook()

```

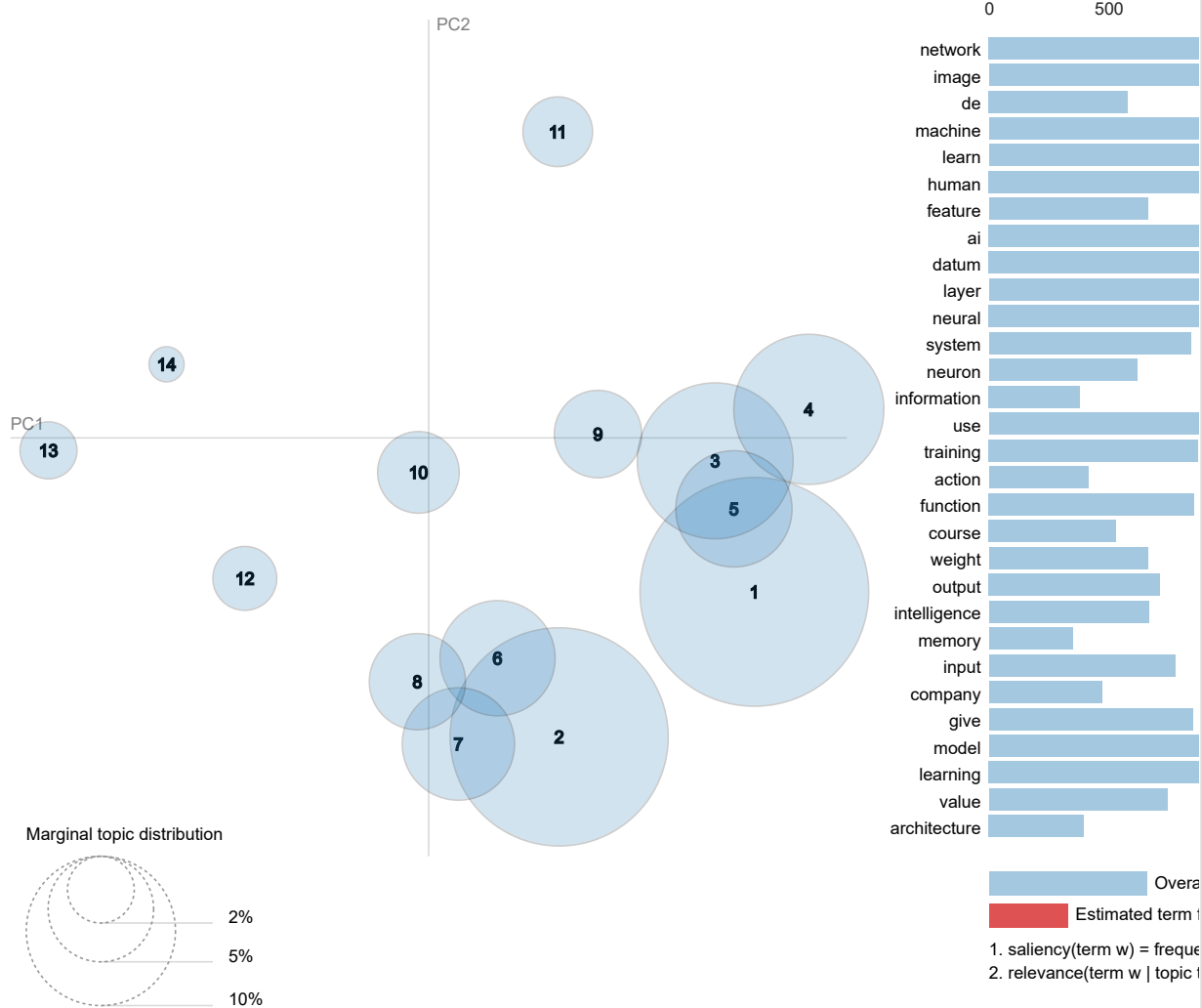
```
vis = gensimvis.prepare(lda_model, corpus, id2word)
vis
```

Selected Topic:

Slide to adjust relevance

 $\lambda = 1$

Intertopic Distance Map (via multidimensional scaling)

Start coding or [generate](#) with AI.

